Informatica® Metadata Command Center
July 2026

Custom Metadata Integration Reference

Informatica Metadata Command Center Custom Metadata Integration Reference
July 2026

© Copyright Informatica LLC 2026

This software and documentation are provided only under a separate license agreement containing restrictions on use and disclosure. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC.

U.S. GOVERNMENT RIGHTS Programs, software, databases, and related documentation and technical data delivered to U.S. Government customers are "commercial computer software" or "commercial technical data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, duplication, disclosure, modification, and adaptation is subject to the restrictions and license terms set forth in the applicable Government contract, and, to the extent applicable by the terms of the Government contract, the additional rights set forth in FAR 52.227-19, Commercial Computer Software License.

Informatica, Informatica Cloud, Informatica Intelligent Cloud Services, PowerCenter, PowerExchange, and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Portions of this software and/or documentation are subject to copyright held by third parties. Required third party notices are included with the product.

The information in this documentation is subject to change without notice. If you find any problems in this documentation, report them to us at infa_documentation@informatica.com.

Informatica products are warranted according to the terms and conditions of the agreements under which they are provided. INFORMATICA PROVIDES THE INFORMATION IN THIS DOCUMENT "AS IS" WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING WITHOUT ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND ANY WARRANTY OR CONDITION OF NON-INFRINGEMENT.

Publication Date: 2026-07-24

Table of Contents

Preface. 4

Chapter 1: Custom metadata integration. 5

Workflow for custom metadata integration. 6

Step 1. Create a custom model. 6

Step 2. Update the custom model definition file. 7

Step 3. Import and publish the custom model. 13

Step 4. Create a custom catalog source type. 14

Step 5. Prepare the custom metadata source. 15

Step 6. Create the custom catalog source. 19

Step 7. Run the custom catalog source. 21

Example: Ingest metadata from Microsoft Access database. 22

Define the custom model. 22

Update the metadata definition files. 29

Create custom lineage. 31

Preface

Read *Custom Metadata Integration* to learn how to ingest custom metadata from any source system into the catalog and create custom catalog sources in Metadata Command Center.

CHAPTER 1

Custom metadata integration

Custom metadata is metadata that you define for a source system for which Metadata Command Center does not provide connectors by default. Custom metadata integration is ingesting custom metadata from any source system into the catalog. Such source systems are called custom source systems.

You can configure a custom catalog source to load metadata into the catalog using the following sources:

- Metadata definition files
- Cloud Data Integration
- Java SDK

Preview Notice: Effective in the July 2023 release, loading metadata into the catalog from Cloud Data Integration and Java SDK are available for preview. Preview functionality is supported for evaluation purposes but is unwarranted and is not supported in production environments or any environment that you plan to push to production. Informatica intends to include the preview functionality in an upcoming release for production use, but might choose not to in accordance with changing market or technical circumstances. For more information, contact Informatica Global Customer Support.

To learn how to load metadata into the catalog from Cloud Data Integration, refer to the following tutorial: Load custom metadata using Data Integration in Metadata Command Center

To learn how to load metadata into the catalog using Java SDK, refer to the following tutorial: Load custom metadata using Java SDK in Metadata Command Center

To ingest custom metadata from a source system, perform the following high-level tasks:

Create a custom model or reuse a system model

A model defines the structure of metadata stored in the catalog. It contains classes, associations, and attributes of the metadata. By default, Metadata Command Center provides models for multiple source systems from which you can extract metadata. These models are called system models. However, to ingest metadata from source systems for which Metadata Command Center doesn't provide a system model, you need to first define a custom model.

Use a custom model to load metadata from any source system into the catalog to meet your business requirements. For example, you are a data analyst in your organization and you need to prepare a report on the annual sales on your e-commerce website. The data that is required to prepare the report is stored in Microsoft Access Database and you are expected to ingest the data from Microsoft Access Database into Data Governance and Catalog. Since Metadata Command Center does not provide a model for Microsoft Access Database, you can create a custom model to ingest metadata from this specific source system into the catalog.

Create a custom catalog source type

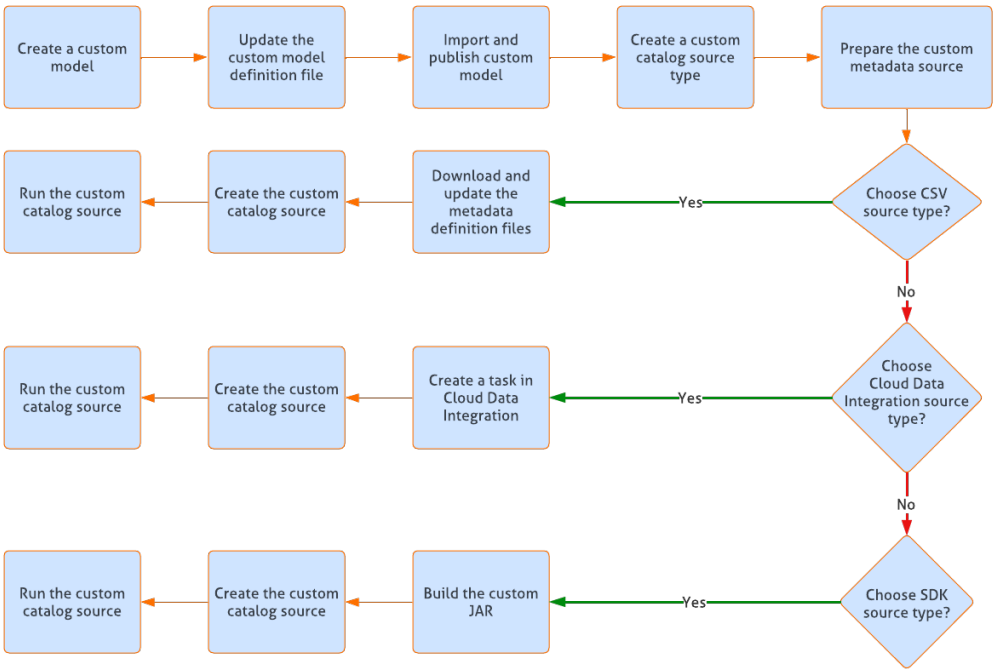
A custom catalog source type represents the type of custom source system from which you want to ingest the metadata. After you define the custom model or reuse a system model, create a custom catalog source type.

Create a custom catalog source

Based on the custom catalog source type, create a custom catalog source. When you create the custom catalog source, provide the details of the metadata that you want to ingest from the source system into the catalog.

Workflow for custom metadata integration

To ingest metadata from a source system, verify whether you can reuse a system model. For example, to ingest metadata from any relational source system, you can use the relational system model included in Metadata Command Center. If you cannot reuse a system model available in Metadata Command Center, perform the following steps to integrate custom metadata into the catalog.



Step 1. Create a custom model

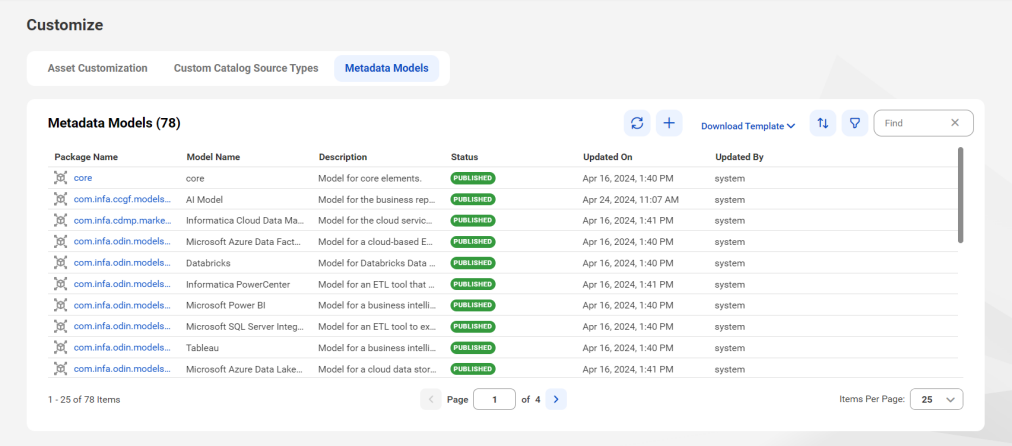
When you create a custom model for custom metadata integration, you configure the metadata that you want to ingest in the catalog. You can also configure options to search, filter, and sort the custom metadata ingested into the catalog.

Create a custom model in one of the following ways:

- Reuse a system model and update the model definition to create and define a custom model. For example, to create a custom model for a relational source system, you can use the `com.infa.odin.models.relational` model included in Metadata Command Center.
- Create a model on your own by using the template model definition file that is available for download in Metadata Command Center.

To create a custom model, perform the following steps:

1. On the **Customize** page, go to the **Metadata Models** tab.



2. To use an existing model that Metadata Command Center provides by default, hover your mouse on the model of your choice and click the **Download Model** icon on the far right.

   The model definition file in the JSON format is downloaded to your machine. The file contains the package name, properties, classes, attributes, and associations of the objects in the source system.

3. Alternatively, click the **Download Template** menu and choose **Model**.

   The sample model file, `sample.json`, is downloaded to you machine. This file contains sample classes, attributes, and associations that you want to extract from the custom source system.

After downloading the sample model or an existing model, update the details of the metadata following the template structure. Then, import the custom model and publish it in Metadata Command Center.

Step 2. Update the custom model definition file

In the model definition JSON file, update the package details, parent classes, subclasses, attributes and associations of the objects in the custom source system based on your business requirements.

Open the JSON file in a text editor of your choice and update the following components of the model file:

Package

The name of the custom model. For example, `com.example.accessdb`. The package represents a container for the classes in the model. Verify that the package name does not contain `com.infa.odin`, `com.informatica` or `infa`. These keywords are reserved for system models.

Required Packages

A comma-separated list of required or reused package names. Use the fully-qualified package names if you are reusing any components such as classes, attributes, or associations from other packages. Verify that you define the name of the package as core if you are reusing other packages. For example, `core`.

Classes

A class describes a group of objects that share the same characteristics. Each class has a set of attributes, those that belongs to itself or from its super classes. It represents the metadata objects that you want to extract from the custom source system. For example, `AccessSchema` is the root class for a model that you define for the Microsoft Access Database source system. `AccessTable` and `AccessView` are the child classes of the `AccessSchema` class. For information about the class properties that you need to define in the model, see "Add class properties" on page 8.

Attributes

An attribute describes the characteristics of an object. Define the model attributes and how they apply to classes and relationship by creating attributeClass entries and attributeRelationship entries. For information about the attribute properties that you need to define in the model, see "Add attribute properties" on page 9.

Class attributes

A class attribute connects attributes to classes and defines how those attributes behave for each class. Use class attributes to define which attributes are available for each class in your model. You can control attribute behavior through properties. Properties can include whether the attribute is required, curatable by users, or hidden from the user interface. When you define a model for a Microsoft Access Database source system, you can assign the `Name` and `DateCreated` attributes to the `AccessTable` class. Specify that `Name` is required while `DateCreated` is optional. Each class attribute entry creates a binding between one class and one attribute with specific rules for that combination. For information about the class attribute properties that you need to define in the model, see "Add classAttribute properties" on page 11.

Associations

An association represents the relationship between two objects in the catalog. You can create associations between the objects within the custom catalog source and to objects already synced to the catalog. For example, for a model that you define for the Microsoft Access Database source system, the columns in the `AccessTable` schema has an association with columns in an Oracle database. For information about the association properties that you need to define in the model, see "Add association properties" on page 11.

Association attributes

An association attribute connects attributes to associations and defines how those attributes apply to relationships between objects. It adds metadata properties to the associations and captures information about the relationship itself, such as when it was established or the order of columns in a table. Each entry binds one association to one attribute and applies specific rules for that combination. You don't need to update association attributes.

Data Type

The data type defines the types of values possible for an attribute. For example, string and integer. For information about the data types that you need to define in the model, see "Add data types" on page 12.

Add class properties

Define the following class properties in the `classes` component of the custom model JSON file.

Property	Description
name	The name of the class.
label	The text displayed to the user to describe this entity.
externalLabel	Not in use.*
description	The description of the class displayed to the user.
isFirstClass	Use this property to prioritize the class in search. Set to true or false. If set to true, Metadata Command Center displays the object count of the classes after you run the calalog source.
IsAbstract	Use this property to specify whether the class is abstract or not. Set to true or false.

Property	Description
cdc	Use this property to specify if the class participates in change data capture. Change data capture is used to record the applied changes for audit purposes. Set to true or false.
superClasses	The list of classes with which the class has a semantic relationship.
deprecated	Use this property to specify whether the class is deprecated. Set to true or false.
indexType	Use this property to specify whether the class can be indexed or not. Specify any of the following values: - FULL. Indexed in both the elastic and graph stores. - FULL_TEXT. Indexed in elastic, but not in the graph store. - NONE. Not indexed.
appendOnly	Not in use.*
extensions	Indicates extension classes. Extension classes are classes that are linked to other classes. Multiple classes can have the same extension class. It contains common attributes linked to different master classes.
softDeleted	Not in use.*
system	Classes indicated as system do not inherit any property from the IClass abstract class. System classes are organized at the same level as IClass.
* The properties that are not in use can be ignored.	

Add attribute properties

Define the following attribute properties in the `attributes` component of the custom model JSON file.

Property	Description
name	The name of the attribute.
label	The text displayed to the user to describe this attribute.
dataType	The data type of the attribute.
description	The description of the attribute displayed to the user.
multivalued	Use this property to specify if the attribute can have multiple values. Set to true or false.
deprecated	Use this property to specify whether the attribute is deprecated or not. Set to true or false.
derived	Use this property to specify if this attribute is derived from an existing attribute.
data	Use this property to specify whether its metadata is derived from Data. This property is used to handle sensitive attributes like value frequencies or data patterns. Set to true or false.

Property	Description
custom	Use this property to specify if the class should be marked as searchable. Set to true or false.
reference	The reference attribute references a registered association and its target class as a native data type of the source class. Use this property to specify the association type to relate the class from which the attribute needs to be projected based on the projectionCondition attribute value.
referencedAssociationAttributes	Not in use.*
referencedAttributes	Use this property to specify projected attributes.
projectionType	Use this property to specify the attribute type to be projected to other classes. Specify any of the following values: - PRIMITIVE. Project a single attribute - NESTED. Project a nested attribute Define the projected attribute in referencedAttributes.
projectionExpressions	The condition to be applied to project an attribute.
defaultValues	The value to be applied if the user does not provide values for ingestion.
isSystem	Use this property to specify whether system attributes should be added for every object or association that is ingested. Set to true or false.
embedded	Use this property to specify whether the reference data type is embedded into the same class as a struct or created as an extension table. Set to true or false.
searchConfiguration	Use this property to specify the search configuration for indexing before data is ingested. Specify any of the following values: - AttributeType. Indicates if the attribute is searchable or viewable. Only searchable attributes are indexed and can be searched. **Note:** You can change the attribute type from VIEWABLE to SEARCHABLE. - FieldMappingTemplate. Elasticsearch uses dynamic mapping to infer the data type of each field and assigns a field type to store each field. Use this property to map attributes in the field mapping template and assign a type to each field. - Suggestable. Suggests similar looking terms based on the provided text. - Aggregatable. Enables aggregate functions on search operations. - Sortable. Enables sorting on search results.
projectionCondition	The condition to project the attribute. You can define conditions in the projectionExpressions and expressionContext attributes.
customizations	Indicates whether you can customize the attribute or not.
deleted	Indicates if the attribute is deleted. You can't use deleted attributes. Use this property to track deleted attributes.
* The properties that are not in use can be ignored.	

Add classAttribute properties

Use classAttribute properties to define how attributes are assigned to classes or objects, such as tables or columns.

The following table describes the classAttribute properties that you define in the `classAttributes` component of the custom model JSON file:

Property	Description
className	The fully qualified name of the class to which the attribute is assigned.
attributeName	The fully qualified name of the attribute being assigned to the class.
isRequired	Specifies whether the attribute must have a value when you create or update an object of this class. Set to true or false. If set to true, the attribute is required.
cdc	Specifies whether the classAttribute participates in change data capture. Change data capture is used to record the applied changes for audit purposes. Set to true or false.

Add association properties

Define the following association properties in the `associations` component of the custom model JSON file.

Property	Description
name	The name of the association.
label	The text displayed to the user to describe the association.
description	The description of the association displayed to the user.
fromClass	The source class of the association.
toLabel	The text that describes the relationship to the target class.
fromLabel	The text that describes the relationship from the source class.
toClass	The target class of the association.
associationKinds	The type of relationship or association between objects in the source system.
deprecated	Use this property to specify whether the association is deprecated or not. Set to true or false.
cdc	Use this property to specify if the association participates in change data capture. Change data capture is used to record the applied changes for audit purposes. Set to true or false.
unidirectional	Indicates if the association is only valid for one direction. Set to true or false.
aggregate	Not in use.*
cardinality	Use this property to specify the cardinality. The value can be OneToOne and OneToMany.

Property	Description
index	Use this property to specify the index type for the association. Set to COLLECTION to configure a single elastic document as a collection for classes that are not prioritized using the isFirstClass property.
customizable	Indicates whether you can customize the association or not. Set to true or false.
custom	Indicates whether the association is customized at least once or not. Set to true or false.
deleted	Indicates if the association is deleted. You can't use deleted associations. Use this property to track deleted associations.
* The properties that are not in use can be ignored.	

Add data types

You can define the data types in a custom model using the following list of core types available in Metadata Command Center:

- STRING
- BOOLEAN
- DATE
- DECIMAL
- RICHTEXT
- INTEGER
- PAIR
- LONG
- Reference Class
- DOUBLE
- DATETIMETZ
- MAP<STRING,CORETYPE>

In the following example, the data type, `curationStatusEnum`, is defined using the core type, `STRING`. Similarly, you can define any data type in a model using the available core types.

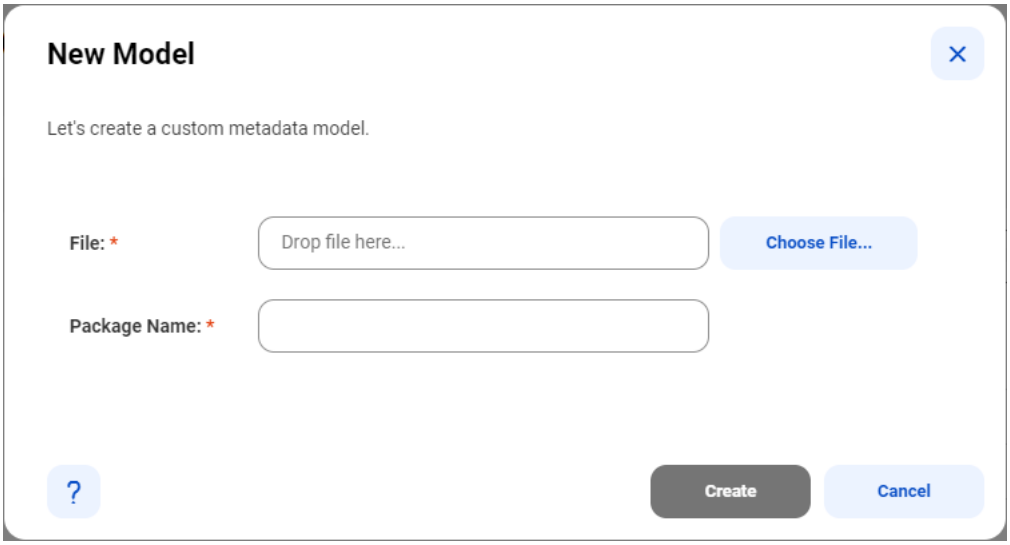
```
{
"name": "curationStatusEnum",
"constraint": {
"constraintType": "LIST_OF_VALUES",
"values": [
"AUTO_ACCEPTED",
"ACCEPTED",
"REJECTED",
"NONE"
]
},
"coreType": "STRING",
"deprecated": false
}
```

Step 3. Import and publish the custom model

After you have created and defined the model for the custom metadata in the JSON file, you can import and publish the custom model. Based on the custom model, Metadata Command Center organizes the storage of the custom metadata that is ingested. To import and publish the custom model, perform the following steps:

1. Click **New** in the left navigation panel.
2. In the **New** dialog box, select **Customization** from the list in the left pane, and click **Metadata Model** on the right pane.

   You can also click the plus icon on the **Metadata Models** page to create a new model.

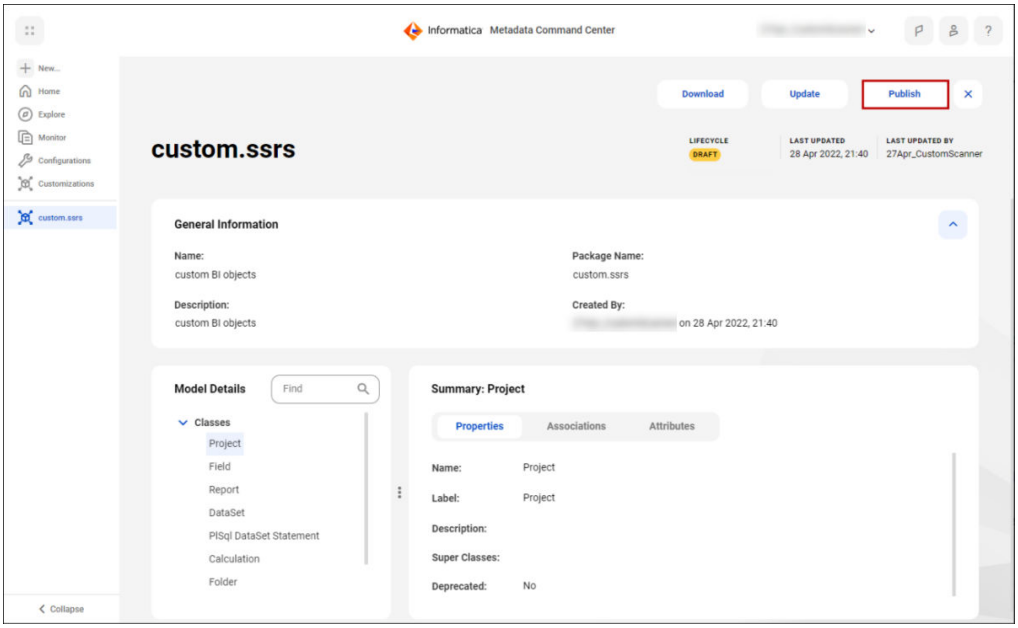


3. In the **New Model** window, click **Choose File** to upload a JSON-based model definition file that you have created for the custom metadata.
4. Enter a package name for the model.

   Verify that the package name is the same as defined in the model definition JSON file.
5. Click **Create**.

   This creates a draft version of the model that appears on the **Metadata Models** page.

6. From the **Metadata Models** page, select the model that you imported. You can view the model details including the classes, attributes, and associations that you defined in the model.



7. Verify that your model appears as you expected. To make any updates to the model, modify the JSON file on your machine and click **Update** to import the latest version of the model file.

   Note: You cannot modify the package name once you have imported the model.

8. To publish the model, click **Publish**.

   The Model Publish job is triggered. You can click **View Status** to monitor the exact status of the job on the Job Monitoring Overview page of that job. After the publishing is successful, the lifecycle of the model changes from Draft to Published. Note that you cannot import an updated model while the model publish job is in progress.

Note the following points about the lifecycle of a model:

- If you update a custom model after it is published, the lifecycle of the model changes to **Published with Draft Available**. You can publish the model again or switch the lifecycle of the model from **Draft** to **Published** to access the published version. By default, the **Draft** version of the model is displayed.
- You cannot discard a model if the lifecycle of the model is in the published state. You can discard only the draft version of the model.
- You cannot modify or delete system models.

Step 4. Create a custom catalog source type

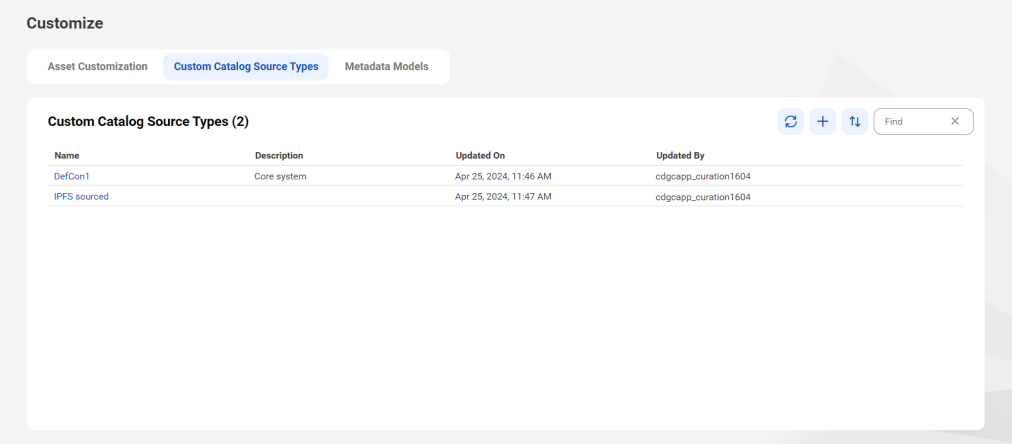
The custom catalog source type represents the custom source system from which you want to ingest the metadata. By default, Metadata Command Center provides connections for a variety of source systems. To ingest metadata from source systems for which there are no predefined catalog source types available in Metadata Command Center, create a custom catalog source type based on which you can then create a custom catalog source.

To create a custom catalog source type, define appropriate roles and assign the Create, Read, Update, and Delete permissions for the **Custom Catalog Source Type** asset for that role in Administrator. For more information about asset permissions that the organization administrator can configure for user roles, see *Asset permissions* in the Administrator help.

To create a custom catalog source type, perform the following steps:

1. Click **New** in the left navigation panel.
2. In the **New** dialog box, select **Customization** from the list of asset types in the left pane, and click **Custom Catalog Source Type** on the right pane.
3. Enter a name that describes the source system from which you want to extract metadata.
4. Optionally, enter a description of the source system and click **Save**.

   The new custom catalog source type appears in the list of custom catalog sources types on the **Customize** page.



5. To modify the name or description of the catalog source type, click the name of the catalog source type, update the name or description of the catalog source type and click **Save**.

You're now ready to create a custom catalog source based on the custom catalog source type.

Step 5. Prepare the custom metadata source

To load metadata from a custom source system, prepare the custom metadata source based on the metadata source type. You can choose to load metadata into the catalog using CSV files, Cloud Data Integration, or Java SDK.

If you choose to use CSV files as the custom metadata source, download and update the metadata definition files.

If you choose to use Cloud Data Integration as the custom metadata source, create and run a mapping task or a linear taskflow in Cloud Data Integration. CSV files that contain metadata are generated from a mapping task or a linear taskflow in Cloud Data Integration.

If you choose to use Java SDK as the custom metadata source, build a custom JAR, and copy it to any location on the Secure Agent machine.

Preview Notice: Effective in the July 2023 release, loading metadata into the catalog using Cloud Data Integration and Java SDK are available for preview. Preview functionality is supported for evaluation purposes but is unwarranted and is not supported in production environments or any environment that you plan to push to production. Informatica intends to include the preview functionality in an upcoming release for production use, but might choose not to in accordance with changing market or technical circumstances. For more information, contact Informatica Global Customer Support.

Load metadata into the catalog using CSV files

Before you create the custom catalog source, download the metadata definition files and enter the details of the metadata that you want to ingest from the custom source system into the metadata files.

Download and update the metadata files template. In the file, add the object details of the specific classes that you defined in the custom model for this source system. Metadata Command Center uses the details entered in the metadata files to load metadata from the custom source system into the catalog.

1. Go to the **Customization** page and click the **Metadata Models** tab.
2. Click the custom model that you created in "Step 1. Create a custom model" on page 6.
3. Click **Download Template** > **Metadata**.
   This downloads the metadata template in the ZIP format to your machine. The ZIP file might contain multiple CSV files for the metadata that you have defined in the custom model.
4. Extract the CSV files included in the ZIP file.
   The ZIP file contains the following CSV files:
   - Object files. The ZIP file might contain multiple object files depending on the number of classes defined in the custom model. The object files are in the following format: `<package name>.<class name>`. You can map each object file to its class in the custom model. Use these files to enter details of objects in the custom source system.
   - links.csv file. This file contains association details between the objects in the source systems. You can associate objects within the source system and define relationships to objects in external source systems. Verify that the metadata file template contains only one links.csv file.

   Open the CSV files in a text editor and enter the objects, association, and lineage details.
5. Enter the details of each object in separate CSV files. Each object corresponds to a specific class that you define in the custom model.
   The following table lists the details that you need to enter in the CSV file for each object of the source system:

Header Field	Description
core.externalId	Required. Unique identity of the object. The ID cannot contain a comma.
core.reference	Optional. Set to true if you want to use this object as a reference asset. Reference assets are used in place of actual assets to view the data lineage among one or more sources that don't exist in the catalog. For more information about using reference assets for custom lineage, see "Create custom lineage" on page 31.
core.assignable	Optional. Set to true if you want this object to be available for connection assignment. If set to true, you can assign connections to this object to view the complete lineage.
core.name	Required. Name of the object.
core.description	Optional. Description of the object.
core.businessName	Optional. Specify the name of the business term that you want to associate with this object.
core.businessDescription	Optional. Specify the description of the business term that you want to associate with this object.

6. Define associations between the objects in source system in the links.csv file that is included in the template. Define all associations between the objects or classes that you defined in the custom model for this source system. The type of associations that you specify depends on the type of associations defined in the `associationKinds` property in the custom model. Create each entry on a separate line.

The following table lists the association details that you need to enter in the `links.csv` file:

Header Fields	
Source	Required. Unique external ID of the source object in the association. Specify the ID of the object in a parent-child format and verify that the ID of the object matches the external ID of the object provided in the objects CSV file. For example, if the source object is a view, then the identity has the full path to the view, that is, `Schema/Table/Column/View`. **Note:** `$resource` represents the catalog source. For this source object, the target object is the root class that is defined in the model. This creates a parent-child association between the specified catalog source and the root class or object in the source system.
Target	Required. Unique external ID of the target object in the association. Specify the ID of the object in a parent-child format and verify that the ID of the object matches the external ID of the object provided in the objects CSV file. For example, if the target object is a table, then the identity has a full path to the table, that is, `Schema/Table`.
Association	Required. The name of the association in the source system. Specify the association for objects in the `<package name>.<association name>` format.

7. Include all the CSV files and create a ZIP file.

Load metadata into the catalog using Cloud Data Integration

To load metadata from Cloud Data Integration, create a mapping task in Cloud Data Integration, run the task, and verify that the mappings generate CSV files.

Verify that the name of the CSV file contains the package name of the metadata model and the component class in the following format: `<Package name>.<Class name>.csv`. For example, the CSV file generated from a metadata model with the package name `com.infa.model.relational` for the class Table is `com.infa.model.relational.Table.csv`.

Note: The header fields in the CSV files can contain an underscore (_).

Load metadata into the catalog using Java SDK

To load metadata into the catalog using Java SDK, build a custom JAR, and copy it to any location on the Secure Agent machine. Informatica provides an Oracle, a Swagger, and a Denodo sample project to help you build custom JARs.

To build a custom JAR, perform the following steps:

1. Download a sample project ZIP file from the following location: `<Informatica Secure Agent installation directory>/apps/Metadata_Foundation_Agent/<version>/data/scanner/custom/`

   Note: Informatica provides several sample project ZIP files. Choose the ZIP file that is most suitable for your use case.

   - For the Oracle sample project, use the sample-custom-sdk-scanner-oracle-xxxx.zip file.
   - For the Swagger sample project, use the sample-custom-sdk-scanner-swagger-xxxx.zip file.
   - For the Denodo sample project, use the sample-custom-sdk-scanner-denodo-xxxx.zip file.

   Here, xxxx represents the bundle version.
2. Copy and extract the ZIP file to a machine that has an integrated development environment (IDE).

3. Import the sample project to a suitable IDE.
   To build the custom JAR with the Denodo sample project, copy the denodo-vdp-jdbcdriver.jar to the lib folder of the extracted ZIP file. The Denodo JAR is available in the following location: `<Denodo installation directory>/tools/client-drivers/jdbc/`
4. Run the `gradle clean build` command.

   Note: If you want to use a build tool other than Gradle, use the custom-scanner-sdk-contract-xxxx.jar file present in the lib folder of the extracted ZIP file. Add the custom-scanner-sdk-contract-xxxx.jar file to the project class path.
5. To build the custom JAR with a sample project, use the Java file located in the src folder of the extracted ZIP file.
   To build the custom JAR with the Oracle sample project, use the CustomOracleScanner.java file located in the src folder of the extracted ZIP file.

   To build the custom JAR with the Swagger sample project, use the CustomSwaggerScanner.java file located in the src folder of the extracted ZIP file.

   To build the custom JAR with the Denodo sample project, use the CustomDenodoScanner.java file located in the src folder of the extracted ZIP file.
6. Implement the code in Java to ingest metadata from the custom source system.
7. Build the JAR based on your requirements.
   You must specify the main class to be executed in the JAR. You can either include all the dependent classes in the JAR and build a fat JAR, or specify class paths in the manifest file to include any external libraries provided outside of the main JAR.

   Note: Use Java Standard Edition version 17, 11, or 8 to build JARs. Newer Java versions will be supported in future releases.

   Important: Informatica is not responsible for any security vulnerabilities that might be associated with the custom JAR.
8. Copy the JAR to any location on the Secure Agent machine.

Consider the following points when you build custom JARs using the Swagger sample project:

- The Swagger model file is provided as part of the Swagger sample project and is available in the following location:
  `sample-custom-sdk-scanner-swagger\src\main\resources\swagger.api.json`
- You must import and publish the Swagger model file in the **Metadata Models** page.
- When you create a custom catalog source for a custom JAR, the name of the configuration parameter must be "Swagger Path". The custom catalog source can extract metadata from multiple JSON files placed in different folders. The value can be the path to a Swagger parent folder that contains the JSON Swagger files or to a JSON Swagger file on the Secure Agent machine.
- You can build custom JARs only from JSON Swagger files.
- The Swagger sample project uses the swagger-parser-2.0.33 library validated on the OpenAPI 3.0 version. You can also update the Swagger parser library version and build custom JARs with newer OpenAPI versions.

Guidelines and best practices to build custom JARs

Consider the following rules and guidelines when you build custom JARs:

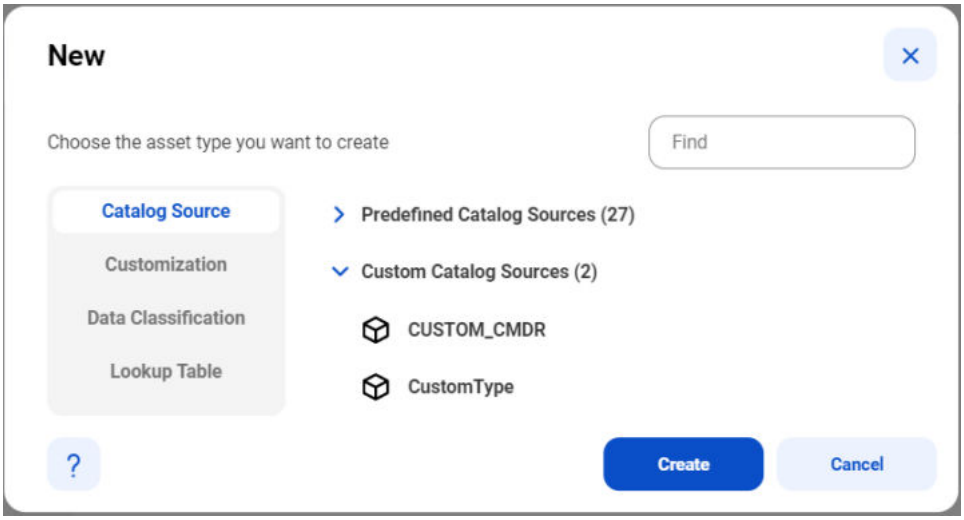
- Use Gradle build tool version 6.8.1.
- Use only the `yyyy-MM-dd'T'HH:mm:ss` date format in the Java file to extract and view metadata of system attributes in Data Governance and Catalog.

Step 6. Create the custom catalog source

After you import and publish a custom model and define the details of your metadata in the metadata files, create a custom catalog source based on the source type.

To create a custom catalog source, define appropriate roles and assign the Create, Read, Update, and Delete permissions for the **Custom Catalog Source Type** asset for that role in Informatica Intelligent Cloud Services Administrator. For more information about asset permissions that the organization administrator can configure for user roles, see *Asset permissions* in the *Administration* help.

1. Click **New** in the left navigation panel and select **Catalog Source** from the list of asset types in the left pane.
2. Expand **Custom Catalog Source Type** on the right pane and select the custom catalog source type that you created for your source system.



3. Click **Create**.
4. On the **Registration** page, enter a name and an optional description for the custom catalog source.
5. In the **Connection Information** area, configure the following options:
   - **Metadata Source Type**. Choose **CSV Files** if you want to load metadata using the metadata ZIP file that contains the CSV files. Choose **CDI Task** if you want to load metadata using Cloud Data Integration. Choose **Java SDK** if you want to load metadata using custom JAR files.
   - If you selected **CSV Files** as the metadata source, configure the following options:

Parameter	Description
Source Type	Choose **Upload** if you want to upload the metadata ZIP file from your machine or choose **Provide a Local Path** to provide the path to the ZIP file located in your runtime environment.
File Details	Applicable if you chose to upload the ZIP file. Click **Browse** to select the ZIP file that you created for the custom metadata. Alternatively, you can drag and drop the file in the file box. **Note:** You can't upload a ZIP file that has dots in the file name. The dot that separates the file name from the extension is allowed.

Parameter	Description
Runtime Environment	Applicable if you chose the **Provide a Local Path** option. From the list, select the name of the runtime environment where the metadata file is located.
Secure Agent	Applicable if you chose the **Provide a Local Path** option. Specify the name of the Secure Agent that contains the metadata file.
Path	Applicable if you chose the **Provide a Local Path** option. Specify the absolute path to the ZIP file in the Secure Agent.

- If you selected **CDI Task** as the metadata source, configure the following options:

Parameter	Description
Task	Click **Browse** to select a Cloud Data Integration mapping task or a linear taskflow within a project and folder.
Runtime Environment	Choose the runtime environment that contains the Secure Agent where the metadata file is located.
Secure Agent	Choose the Secure Agent machine that stores the metadata files generated by Cloud Data Integration.
Path	Enter the path to the CSV files on the machine that hosts the Secure Agent. Use the path that you specified when you created the Cloud Data Integration task. **Note:** If you intend to use the same path for subsequent mapping tasks, ensure that you delete the files in the directory.

- If you selected **Java SDK** as the metadata source, configure the following options:

Parameter	Description
Runtime Environment	Choose the runtime environment that contains the Secure Agent where the JAR is located.
Secure Agent	Choose the Secure Agent machine that stores the JAR files.
Executable JAR Path	Enter the absolute path to the executable JAR on the Secure Agent machine.

6. Click **Next** to go to the **Configuration** page.

   On the **Configuration** page, the Metadata Extraction capability is enabled by default.
   If you selected **Java SDK** as the metadata source and want to pass parameters for your custom JAR, add configuration parameters on the **Metadata Extraction** tab. For example, you can add driver class, URL, username, and password as configuration parameters to connect to an Oracle database.

To pass parameters for a custom JAR built using the Swagger sample project, configure the following options in the **Configuration Parameters** area:

Parameter	Description
Name	Enter the name as "Swagger Path".
Value	Enter the path to a Swagger parent folder that contains the JSON Swagger files or to a JSON Swagger file on the Secure Agent machine. The custom catalog source can extract metadata from multiple JSON files placed in different folders.

To pass parameters for a custom JAR built using the Denodo sample project, configure the following options in the **Configuration Parameters** area:

Name	Value
Enter the name as "url".	Enter the URL to access the Denodo source system. Example: jdbc:denodo://<host name>:<port number>/admin By default, the port number is 9999.
Enter the name as "userName".	Enter the username to log in to the Denodo server.
Enter the name as "password".	Enter the password associated with the username.
Enter the name as "schema".	Enter the schema to connect to the Denodo server.

You can enable lineage discovery, data classification, and glossary association for the custom catalog source. For more information about enabling these capabilities, see the corresponding catalog source help.

7. Click **Next** to go to the **Assocations** page.

   Assign permissions to users to access this custom catalog source. For more information about assigning roles and users to technical assets, see the corresponding catalog source help.

8. Click **Next** to go to the Schedule page.

   Optionally, select schedules to run the catalog source job.

9. Click **Save**.

After you save the custom catalog source, it appears in the list of catalog sources on the **Explore** page on the left navigation tab.

Step 7. Run the custom catalog source

When the catalog source runs, Metadata Command Center processes the metadata from the CSV files and ingests the metadata from the source system into the catalog.

You can run the custom catalog source in one of the following ways:

- Click **Run** on the custom catalog source creation wizard and click **Run** again on the **Run Catalog Source Job** dialog box.
- Optionally, select **Run** from the he Action menu on the **Explore** page for the custom catalog source.

A job is triggered to run the catalog source. You can monitor the status of the job on the **Overview** page for that catalog source job.

When the catalog source job is successful, you can search for the custom catalog source in Data Governance and Catalog and view the ingested assets from the custom source system.

Example: Ingest metadata from Microsoft Access database

You are a data analyst in your organization and you need to prepare a report on the annual sales on your e-commerce website. The data that is required to prepare the report is stored in Microsoft Access database and you are expected to ingest the data from Microsoft Access database into Data Governance and Catalog. Since Metadata Command Center does not provide a predefined source system for Microsoft Access database, you can create a custom catalog source to ingest metadata from this specific source system into the catalog.
In this example, let's define a custom model and update the metadata definition files to ingest metadata from Microsoft Access database. For the complete workflow of ingesting metadata from a custom source system, see "Workflow for custom metadata integration" on page 6.

Define the custom model

To create a custom model, you can either reuse a system model or download the template model definition file. See "Step 1. Create a custom model" on page 6.

In the custom model JSON file for Microsoft Access database, define the appropriate classes, attributes, and associations.

Classes

Define the following classes in the custom model for Microsoft Access database:

Class	Type
AccessSchema	Schema
AccessTable	Table
AccessTableColumn	Column
AccessView	View
AccessViewColumn	View column

Attributes

Define the following attributes in the custom model for Microsoft Access database:

Attribute	Data Type
Name	String
Abbreviation	String

Attribute	Data Type
Business Usage	String
DateCreated	String
Datatype	String
Length	String

Associations

Define the following attributes in the custom model for Microsoft Access database:

Association	Association Type
AccessSchemaAccessTable	Parent-child
AccessTableAccessTableColumn	Parent-child
AccessSchemaAccessView	Parent-child
AccessViewAccessViewColumn	Parent-child

Sample custom model JSON file

The following JSON file is the sample custom model for Microsoft Access database. It defines classes, attributes, and associations for the Microsoft Access database from which you want to extract metadata:

```
{
    "packageName": "relational.accessdb",
    "packageLabel": "Access DB Objects",
    "packageDescription": "Model for Access DB Objects",
    "deprecated": false,
    "attributes": [{
            "name": "Name",
            "label": "Name1",
            "dataType": "core.String",
            "description": "",
            "multivalued": false,
            "deprecated": false,
            "derived": false,
            "data": false,
            "isCustom": false,
            "defaultValues": [],
            "isSystem": false,
            "embedded": false,
            "searchConfiguration": {
                "suggestable": false,
                "aggregatable": false,
                "sortable": false,
                "attributeType": "VIEWABLE"
            }
        },
        {
            "name": "Abbreviation",
            "label": "Abbreviation",
            "dataType": "core.String",
            "description": "",
            "multivalued": false,
            "deprecated": false,
            "derived": false,
            "data": false,
```

```
        "isCustom": false,
        "defaultValues": [],
        "isSystem": false,
        "embedded": false,
        "searchConfiguration": {
            "suggestable": false,
            "aggregatable": false,
            "sortable": false,
            "attributeType": "VIEWABLE"
        }
    },
    {
        "name": "BusinessUsage",
        "label": "Business Usage",
        "dataType": "core.String",
        "description": "",
        "multivalued": false,
        "deprecated": false,
        "derived": false,
        "data": false,
        "isCustom": false,
        "defaultValues": [],
        "isSystem": false,
        "embedded": false,
        "searchConfiguration": {
            "suggestable": false,
            "aggregatable": false,
            "sortable": false,
            "attributeType": "VIEWABLE"
        }
    },
    {
        "name": "DateCreated",
        "label": "Date Created",
        "dataType": "core.String",
        "description": "",
        "multivalued": false,
        "deprecated": false,
        "derived": false,
        "data": false,
        "isCustom": false,
        "defaultValues": [],
        "isSystem": false,
        "embedded": false,
        "searchConfiguration": {
            "suggestable": false,
            "aggregatable": false,
            "sortable": false,
            "attributeType": "VIEWABLE"
        }
    },
    {
        "name": "Datatype",
        "label": "Datatype",
        "dataType": "core.String",
        "description": "",
        "multivalued": false,
        "deprecated": false,
        "derived": false,
        "data": false,
        "isCustom": false,
        "defaultValues": [],
        "isSystem": false,
        "embedded": false,
        "searchConfiguration": {
            "suggestable": false,
            "aggregatable": false,
            "sortable": false,
            "attributeType": "VIEWABLE"
        }
    },
```

```
    {
        "name": "Length",
        "label": "Length",
        "dataType": "core.String",
        "description": "",
        "multivalued": false,
        "deprecated": false,
        "derived": false,
        "data": false,
        "isCustom": false,
        "defaultValues": [],
        "isSystem": false,
        "embedded": false,
        "searchConfiguration": {
            "suggestable": false,
            "aggregatable": false,
            "sortable": false,
            "attributeType": "VIEWABLE"
        }
    }
],
"associationAttributes": [],
"associationKinds": [],
"referenceAttributes": [],
"version": 1,
"modifiedBy": "",
"referenceDataTypes": [],
"requiredPackages": [],
"classes": [{
        "name": "AccessSchema",
        "label": "Access Schema",
        "description": "",
        "isFirstClass": true,
        "isAbstract": false,
        "superClasses": [
            "core.DataSource"
        ],
        "deprecated": false,
        "indexType": "FULL",
        "appendOnly": true
    },
    {
        "name": "AccessTable",
        "label": "Access Table",
        "description": "",
        "isFirstClass": true,
        "isAbstract": false,
        "superClasses": [
            "core.DataSet"
        ],
        "deprecated": false,
        "indexType": "FULL",
        "appendOnly": true
    },
    {
        "name": "AccessTableColumn",
        "label": "Access Table Column",
        "description": "",
        "isFirstClass": true,
        "isAbstract": false,
        "superClasses": [
            "core.DataElement"
        ],
        "deprecated": false,
        "indexType": "FULL",
        "appendOnly": true
    },
    {
        "name": "AccessView",
        "label": "Access View",
        "description": "",
```

```
            "isFirstClass": true,
            "isAbstract": false,
            "superClasses": [
                "core.DataSet"
            ],
            "deprecated": false,
            "indexType": "FULL",
            "appendOnly": true
        },
        {
            "name": "AccessViewColumn",
            "label": "Access View Column",
            "description": "",
            "isFirstClass": true,
            "isAbstract": false,
            "superClasses": [
                "core.DataElement"
            ],
            "deprecated": false,
            "indexType": "FULL",
            "appendOnly": true
        }
    ],
    "classAttributes": [{
            "className": "relational.accessdb.AccessSchema",
            "attributeName": "relational.accessdb.Name",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessSchema",
            "attributeName": "relational.accessdb.Abbreviation",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessSchema",
            "attributeName": "relational.accessdb.BusinessUsage",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessSchema",
            "attributeName": "relational.accessdb.DateCreated",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessSchema",
            "attributeName": "relational.accessdb.Datatype",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessSchema",
            "attributeName": "relational.accessdb.Length",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessTable",
            "attributeName": "relational.accessdb.Name",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessTable",
            "attributeName": "relational.accessdb.Abbreviation",
            "isRequired": false,
            "cdc": false
        },
```

```
{
    "className": "relational.accessdb.AccessTable",
    "attributeName": "relational.accessdb.BusinessUsage",
    "isRequired": false,
    "cdc": false
},
{
    "className": "relational.accessdb.AccessTable",
    "attributeName": "relational.accessdb.DateCreated",
    "isRequired": false,
    "cdc": false
},
{
    "className": "relational.accessdb.AccessTable",
    "attributeName": "relational.accessdb.Datatype",
    "isRequired": false,
    "cdc": false
},
{
    "className": "relational.accessdb.AccessTable",
    "attributeName": "relational.accessdb.Length",
    "isRequired": false,
    "cdc": false
},
{
    "className": "relational.accessdb.AccessTableColumn",
    "attributeName": "relational.accessdb.Name",
    "isRequired": false,
    "cdc": false
},
{
    "className": "relational.accessdb.AccessTableColumn",
    "attributeName": "relational.accessdb.Abbreviation",
    "isRequired": false,
    "cdc": false
},
{
    "className": "relational.accessdb.AccessTableColumn",
    "attributeName": "relational.accessdb.BusinessUsage",
    "isRequired": false,
    "cdc": false
},
{
    "className": "relational.accessdb.AccessTableColumn",
    "attributeName": "relational.accessdb.DateCreated",
    "isRequired": false,
    "cdc": false
},
{
    "className": "relational.accessdb.AccessTableColumn",
    "attributeName": "relational.accessdb.Datatype",
    "isRequired": false,
    "cdc": false
},
{
    "className": "relational.accessdb.AccessTableColumn",
    "attributeName": "relational.accessdb.Length",
    "isRequired": false,
    "cdc": false
},
{
    "className": "relational.accessdb.AccessView",
    "attributeName": "relational.accessdb.Name",
    "isRequired": false,
    "cdc": false
},
{
    "className": "relational.accessdb.AccessView",
    "attributeName": "relational.accessdb.Abbreviation",
    "isRequired": false,
    "cdc": false
```

```
        },
        {
            "className": "relational.accessdb.AccessView",
            "attributeName": "relational.accessdb.BusinessUsage",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessView",
            "attributeName": "relational.accessdb.DateCreated",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessView",
            "attributeName": "relational.accessdb.Datatype",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessView",
            "attributeName": "relational.accessdb.Length",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessViewColumn",
            "attributeName": "relational.accessdb.Name",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessViewColumn",
            "attributeName": "relational.accessdb.Abbreviation",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessViewColumn",
            "attributeName": "relational.accessdb.BusinessUsage",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessViewColumn",
            "attributeName": "relational.accessdb.DateCreated",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessViewColumn",
            "attributeName": "relational.accessdb.Datatype",
            "isRequired": false,
            "cdc": false
        },
        {
            "className": "relational.accessdb.AccessViewColumn",
            "attributeName": "relational.accessdb.Length",
            "isRequired": false,
            "cdc": false
        }

    ],
    "associations": [{
            "name": "AccessSchemaAccessTable",
            "label": "AccessSchemaAccessTable",
            "description": "",
            "fromClass": "relational.accessdb.AccessSchema",
            "toClass": "relational.accessdb.AccessTable",
            "fromLabel": "AccessSchema",
            "toLabel": "AccessTable",
```

```
            "associationKinds": [
                "core.ParentChild"
            ],
            "deprecated": false,
            "unidirectional": false,
            "aggregate": false
        },
        {
            "name": "AccessTableAccessTableColumn",
            "label": "AccessTableAccessTableColumn",
            "description": "",
            "fromClass": "relational.accessdb.AccessTable",
            "toClass": "relational.accessdb.AccessTableColumn",
            "fromLabel": "AccessTable",
            "toLabel": "AccessTableColumn",
            "associationKinds": [
                "core.ParentChild"
            ],
            "deprecated": false,
            "unidirectional": false,
            "aggregate": false
        },
        {
            "name": "AccessSchemaAccessView",
            "label": "AccessSchemaAccessView",
            "description": "",
            "fromClass": "relational.accessdb.AccessSchema",
            "toClass": "relational.accessdb.AccessView",
            "fromLabel": "AccessSchema",
            "toLabel": "AccessView",
            "associationKinds": [
                "core.ParentChild"
            ],
            "deprecated": false,
            "unidirectional": false,
            "aggregate": false
        },
        {
            "name": "AccessViewAccessViewColumn",
            "label": "AccessViewAccessViewColumn",
            "description": "",
            "fromClass": "relational.accessdb.AccessView",
            "toClass": "relational.accessdb.AccessViewColumn",
            "fromLabel": "AccessView",
            "toLabel": "AccessViewColumn",
            "associationKinds": [
                "core.ParentChild"
            ],
            "deprecated": false,
            "unidirectional": false,
            "aggregate": false
        }
    ]
}
```

Update the metadata definition files

Before you create the custom catalog source for Microsoft Access database, update the metadata definition files and enter the details of the metadata that you want to ingest from Microsoft Access database. To begin creating the metadata definition files, download and use the metadata files template. See "Step 5. Prepare the custom metadata source" on page 15.

Enter object details

The metadata template ZIP file for Microsoft Access database contains multiple object files in the CSV format. These object files represent the classes that you defined in the custom model. The object file names are in the format *<package name>.<class name>*. For example, `relational.accessdb.AccessSchema`.

Open each CSV file in any text editor, enter the following details for each class, and save the file:

Note that the `core.externalId` and `core.name` are mandatory fields in each file.

Object file	core.externalId	core.name	core.description	relational.accessdb.Name	rela
relational.accessdb.AccessSchema	HERMES	HERMES	This is a schema	HERMES	HRM
relational.accessdb.AccessTable	HERMES/ CUSTOMER	CUSTOMER	This is a table	CUSTOMER	CUS
relational.accessdb.AccessTableColumn	HERMES/ CUSTOMER/ CUSTOMER_ID	CUSTOMER_ID	This is a table column	CUSTOMER_ID	ID
relational.accessdb.AccessView	HERMES/ V_CUSTOMER	V_CUSTOMER	This is a view	V_CUSTOMER	V_CU
relational.accessdb.AccessViewColumn	HERMES/ V_CUSTOMER/ V_CUSTOMER_ID	V_CUSTOMER_ID	This is a view column	V_CUSTOMER_ID	V_CU

Enter association details

Open the `links.csv` file in any text editor. Enter the association details for the objects in Microsoft Access database as shown in the following table, and save the `links.csv` file:

Source	Target	Association
HERMES	HERMES/CUSTOMER	relational.accessdb.AccessSchemaAccessTable
HERMES	HERMES/V_CUSTOMER	relational.accessdb.AccessSchemaAccessView
HERMES/CUSTOMER	HERMES/CUSTOMER/ CUSTOMER_ID	relational.accessdb.AccessTableAccessTableColumn
HERMES/CUSTOMER	HERMES/CUSTOMER/ CUSTOMER_NAME	relational.accessdb.AccessTableAccessTableColumn
HERMES/V_CUSTOMER	HERMES/V_CUSTOMER/ V_CUSTOMER_ID	relational.accessdb.AccessViewAccessViewColumn
$resource	HERMES	core.ResourceParentChild
HERMES/CUSTOMER	HERMES/V_CUSTOMER	core.DataSetDataFlow
HERMES/CUSTOMER/ CUSTOMER_ID	HERMES/V_CUSTOMER/ V_CUSTOMER_ID	core.DirectionalDataFlow

After updating the files, add all the object CSV files and the `links.csv` file in a ZIP file and provide the ZIP file when you create the custom catalog source.

Create custom lineage

Lineage defines the data flow across systems in your organization. You can create custom lineage to view the data lineage information for the assets in your organization. You can do this either by using the `links.csv` file or using reference assets that are derived from reference catalog sources.

Note: When you create a custom catalog source without the core.externalID for an object, the custom catalog source job generates an external ID. This ID appears in the **Reference ID** field on the **System Attributes** tab of the asset in Data Governance and Catalog. You can use this external ID to create custom lineage.

Using the links.csv file

To create custom lineage using the `links.csv` file, define associations between any two existing objects in the catalog. Verify that the `links.csv` file contains the following details for defining associations between objects:

Header Fields	Description
Source	Required. Unique Reference ID of the object in the catalog source that has been synced. You can find the reference ID of the catalog source object in the **Properties** tab in Data Governance and Catalog.
Target	Required. Unique Reference ID of the object in the catalog source that has been synced. You can find the reference ID of the catalog source object in the **Properties** tab in Data Governance and Catalog.
Association	Required. Indicates the type of association. Specify values based on the type of source and target object that you want to link. You can define the following types of associations: - core.ResourceParentChild. To link a parent object to a top-level child object. For example, catalog source to schema. - core.DataSourceParentChild. To link a source system to a data set. For example, schema to table object. - core.DataSetToDataElementParentship. To link a data set to a data element. For example, table to column. - core.DataSetDataFlow. To link source and target objects at the data set level. This association creates data set-level lineage. For example, table to view. - core.DirectionalDataFlow. To link source and target objects at the data element level. This association creates data element-level lineage. For example, table column to view column.

Sample custom lineage file

The following sample shows a `links.csv` file to define lineage between relational objects:

```
Source,Target,Association
06089733190270f1f3938de0b181382c://SQLDB/dbo/
customer_master~com.infa.odin.models.relational.Table,06089733190270f1f3938de0b181382c://
SQLDB/dbo/CUSTOMERS~com.infa.odin.models.relational.Table,core.DataSetDataFlow
06089733190270f1f3938de0b181382c://SQLDB/dbo/customer_master/
custid~com.infa.odin.models.relational.Column,06089733190270f1f3938de0b181382c://
SQLDB/dbo/CUSTOMERS/
CustomerID~com.infa.odin.models.relational.Column,core.DirectionalDataFlow
```

Using reference assets

You can configure predefined or custom catalog sources to view lineage with reference assets from referenced catalog sources.

When you configure a custom catalog source, set the `core.reference` and `core.assignable` header fields to true in the metadata CSV file for each object of the source system. See "Step 6. Create the custom catalog source" on page 19. The source system objects are treated as reference assets and you can assign connections to the custom catalog source in order to define lineage.

After creating the custom catalog source and running the catalog source job, the ingested objects from the referenced catalog source appear as reference assets and Data Governance and Catalog displays the lineage with the reference assets. You can also see the reference catalog source listed on the **Connection Assignment** page in Metadata Command Center. To link the reference assets to actual objects and to view the complete lineage, assign a connection for the reference catalog source.